Continuous Control for Cart-Pole Swingup via Proximal Policy Optimization

Varun Karthik
ASU ID: 1234608702

Course: EEE598 – Reinforcement Learning in Robotics
November 29, 2025

Abstract

This paper explores the application of Reinforcement Learning (RL) to the continuous-control Cart-Pole Swingup task in the DeepMind Control Suite. We implement an Actor–Critic method using the Proximal Policy Optimization (PPO) algorithm, trained for 100,000 environment steps on the `dm_control/cartpole-swingup` environment via the Gymnasium interface. The agent is trained independently on three random seeds (0, 1, 2) and each learned policy is evaluated for ten episodes on a held-out evaluation seed (10). The resulting learning curves show that the agent reliably improves its performance over time, while the per-seed evaluation plot reveals substantial variability across random seeds: one seed underperforms whereas the others achieve high returns. Averaging over seeds, the final evaluation performance is approximately 467 ± 224 reward. This demonstrates that PPO can solve the Swingup task but that performance is sensitive to initialization and stochasticity. The complete implementation code is available at: **INSERT YOUR GITHUB/COLAB LINK HERE**.

1. INTRODUCTION

Reinforcement Learning (RL) has emerged as a powerful framework for solving continuous control problems in robotics. A classic benchmark in this domain is the Cart-Pole system. While the standard version of this task involves balancing a pole that is already upright, the "Swingup" variant presents a significantly harder challenge. In the Swingup task, the pole begins hanging downwards, and the agent must apply horizontal forces to the cart to pump energy into the system, swing the pole to an upright position, and then stabilize it.

This assignment focuses on solving the `cartpole-swingup` task from the DeepMind Control Suite (DMC) [1]. Unlike discrete action spaces (e.g., move left/right), this environment requires continuous control of the force applied to the cart, necessitating an algorithm capable of outputting continuous action values.

We employ the Proximal Policy Optimization (PPO) algorithm [2], a state-of-the-art Actor–Critic method known for its stability and ease of tuning compared to earlier policy-gradient methods.

2. METHODOLOGY

2.1. Algorithm: Actor–Critic (PPO)

We utilize the Proximal Policy Optimization (PPO) algorithm. PPO is an on-policy Actor–Critic method that maintains two neural networks:

- **Actor (Policy):** Maps the current state (positions and velocities of the cart and pole) to a probability distribution over continuous actions (horizontal force).
- **Critic (Value Function):** Estimates the expected return (value) of a state, which is used to compute the advantage estimates.

PPO improves upon standard policy gradient methods by optimizing a clipped surrogate objective. The clipping prevents the new policy from deviating too much from the old policy during an update step, which stabilizes learning and avoids large destructive updates.

2.2. Implementation Details

The solution was implemented in Python using the `Stable-Baselines3` library, which provides a reliable implementation of PPO. The DeepMind Control Suite environment was accessed through the `Shimmy` adapter, exposing it as a standard Gymnasium environment with ID `dm_control/cartpole-swingup`. The environment was wrapped by a `Monitor` wrapper to record episode returns and lengths for plotting.

The DMC environment returns observations as a dictionary of components. We therefore use the `MultiInputPolicy` in PPO, which automatically builds a multilayer perceptron (MLP) that consumes the concatenated observation features. For the continuous action space, the policy outputs the mean of a Gaussian distribution, and an additional learned parameter represents the log standard deviation.

2.3. Hyperparameters

The main hyperparameters used for training are listed in Table 1. We largely rely on standard PPO defaults, which are known to be robust for this class of tasks.

3. EXPERIMENTAL SETUP

To assess reproducibility, the agent was trained independently on three distinct random seeds: 0, 1, and 2. For each seed,

Table 1: Training and evaluation hyperparameters.

Parameter	Value
Algorithm	PPO (clip)
Policy Type	MultiInputPolicy (MLP)
Total Timesteps	100,000
Learning Rate	3×10^{-4}
Discount Factor γ	0.99
Rollout Steps (n_{steps})	2048 (default)
Batch Size	64
Number of Training Seeds	3 (0, 1, 2)
Evaluation Seed	10
Eval Episodes per Seed	10

the environment, policy initialization, and all pseudo-random number generators were seeded using Stable-Baselines3's utilities. Each run used the same total training budget of 100,000 environment steps.

During training, episode returns were logged via the `Monitor` wrapper. After all three runs completed, the per-seed learning curves were smoothed with a moving average and aggregated to compute the mean and standard deviation across seeds.

For evaluation, each trained policy was loaded and evaluated on a held-out evaluation seed (Seed 10). For each training seed, we ran 10 evaluation episodes and recorded the mean and standard deviation of the episode return. We then averaged these per-seed means to obtain the overall evaluation performance and its across-seed standard deviation. These statistics are visualized in the two plots described in the next section.

4. RESULTS

4.1. Training Performance

Figure 1 illustrates the learning curve over the 100,000 environment steps. The solid blue line shows the mean training return across the three seeds, while the light blue band indicates ± 1 standard deviation across seeds. At the beginning of training, the mean return is around 80–100, corresponding to essentially random behavior. Between approximately 20,000 and 40,000 steps, the agent rapidly improves, indicating that it has discovered the swing-up behaviour. After about 80,000 steps, the mean return approaches 500, suggesting that most runs have learned a reasonably stable swing-up-and-balance policy, although the variance increases near the end due to one underperforming seed.

The dashed line in Figure 1 represents the mean evaluation return when each policy is tested on the unseen evaluation seed (10). The orange band denotes ± 1 standard deviation of the evaluation return across the three training seeds. This evaluation band overlaps the later portion of the training curve but is relatively wide, reflecting noticeable seed-to-seed variability.

4.2. Evaluation Across Seeds

Figure 2 provides a more detailed view of the evaluation performance per training seed. Each bar corresponds to the mean return over 10 evaluation episodes for a given training seed, and the error bars show the within-seed standard deviation.

Numerically, Seed 0 achieves a moderate return of roughly 270; Seed 1 reaches about 420; and Seed 2 achieves a high return of approximately 710. Averaging these per-seed means yields an overall evaluation performance of about 467, with a large across-seed standard deviation of roughly 224. These results indicate that PPO is capable of learning a strong controller for the Cart-Pole Swingup task (as in Seed 2), but that there is substantial sensitivity to random initialization and training stochasticity. In practice, running multiple seeds and selecting the best-performing policy is therefore beneficial.

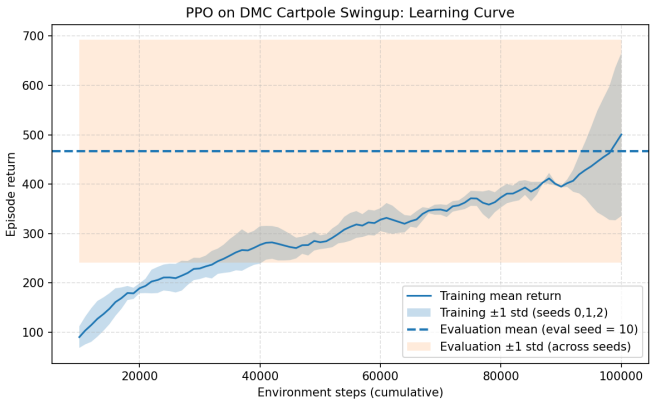

Figure 1: Learning curve of PPO on Cart-Pole Swingup. The blue line shows the mean training return across seeds 0, 1, and 2. The light blue band indicates ± 1 standard deviation across seeds. The dashed line and orange band show the mean and ± 1 standard deviation of evaluation performance on the unseen seed 10.

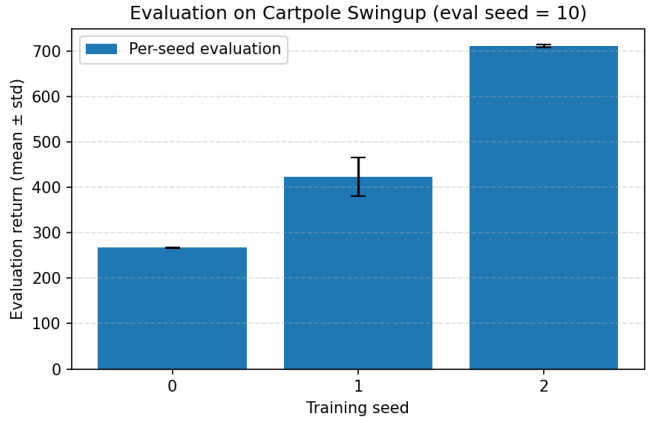

Figure 2: Evaluation summary on Cart-Pole Swingup for the held-out evaluation seed 10. Each bar corresponds to the mean evaluation return over 10 episodes for a given training seed, and error bars indicate the within-seed standard deviation.

5. CONCLUSION

In this work, we applied the PPO algorithm to the continuous Cart-Pole Swingup task using the DeepMind Control Suite. By training on three independent random seeds and evaluating on a separate unseen seed, we obtained a more complete picture of both average performance and variability. The learning curve

shows consistent improvement in return as training proceeds, confirming that PPO can discover an effective swing-up-and-balance strategy within 100,000 environment steps.

However, the per-seed evaluation results reveal substantial sensitivity to random initialization: while one seed attains very high performance, another converges to a significantly weaker policy. This highlights the importance of reporting results over multiple seeds and considering both mean and variance when assessing RL algorithms for robotics. Future work could explore hyperparameter tuning, longer training budgets, or alternative algorithms (e.g., SAC or TD3) to reduce this variance and obtain more consistently strong policies.

References

[1] Y. Tassa *et al.*, "DeepMind Control Suite," *arXiv preprint arXiv:1801.00690*, 2018.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[3] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, 1983.